Q8. Apply non-linear filters (median or other rank filters) to remove noise from data/images provided in OPE7 and perform comprehensive analysis. **CO3, CO4**

Identify noise types present in the dataset (salt-and-pepper, impulse, speckle, etc.).Implement median filtering with varying window sizes and, optionally, other non-linear filters (alpha-trimmed mean, morphological filters).Analyze how filter size and type affect detail preservation vs. noise removal. Observations: optimum filter sizes per noise type, edge preservation analysis, and failure cases. Summarize best-performing methods for each noise type and practical recommendations.

Code:

```
!pip install -q scikit-image opencv-python matplotlib numpy scipy pandas

import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import util, filters, morphology, exposure
from skimage.metrics import peak_signal_noise_ratio as PSNR, structural_similarity as SSIM
from scipy.ndimage import median_filter
import pandas as pd
from skimage.feature import canny

def show(title, img1, img2, l1="Original", l2="Filtered"):
    plt.figure(figsize=(10,4))
    plt.suptitle(title)
    plt.subplot(1,2,1); plt.imshow(img1, cmap='gray'); plt.title(l1); plt.axis("off")
    plt.subplot(1,2,2); plt.imshow(img2, cmap='gray'); plt.title(l2); plt.axis("off")
    plt.show()

def to_float(img):
    if img.dtype == np.uint8:
        return img.astype(np.float32)/255.0
    return img.astype(np.float32)

def to_uint8(img):
    return np.clip(img*255.0,0,255).astype(np.uint8)

# Noise addition
def add_salt_pepper(img, amount=0.05):
    return util.random_noise(img, mode='s&p', amount=amount)

def add_speckle(img, var=0.04):
    return util.random_noise(img, mode='speckle', var=var)
```

```
def add_gaussian(img, var=0.004):
    return util.random_noise(img, mode='gaussian', var=var)

def alpha_trimmed_mean(img, size=3, d=2):
    pad = size//2
    padded = np.pad(img, pad, mode='reflect')
    out = np.zeros_like(img)
    flat_n = size*size
    trim = d//2

    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            block = padded[i:i+size, j:j+size].flatten()
            block_sorted = np.sort(block)
            trimmed = block_sorted[trim:flat_n-trim]
            out[i,j] = np.mean(trimmed)
    return out

def edge_overlap(ref, proc):
    ref_e = canny(ref)
    proc_e = canny(proc)
    inter = np.logical_and(ref_e, proc_e).sum()
    return inter / max(ref_e.sum(),1)

image_paths = [
    "/content/Picture1.png",
    "/content/Picture2.png",
    "/content/Picture3.png",
    "/content/Picture4.png",
    "/content/Picture5.png"
]

images = {}
for p in image_paths:
    img = cv2.imread(p,0)
    if img is not None:
        images[p] = to_float(img)
        print("Loaded:", p)
    else:
        print(" Missing:", p)

if not images:
    raise RuntimeError("No images loaded.")

noisy_versions = {}
```

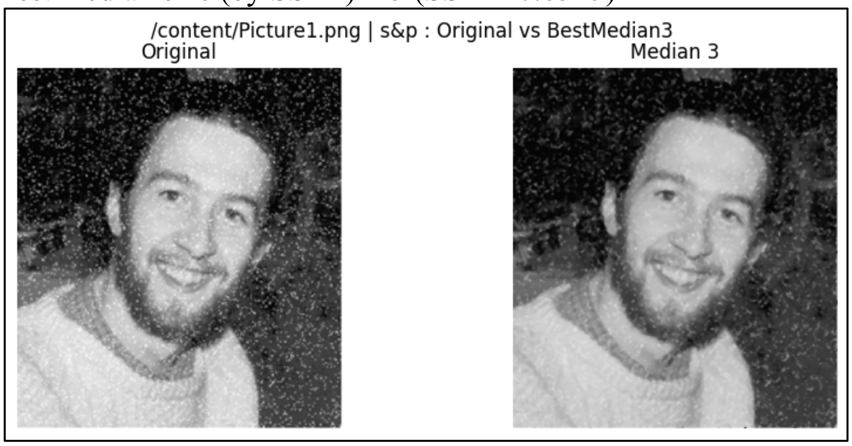
```
for p, img in images.items():

  noisy_versions[p] = {
      "salt_pepper": add_salt_pepper(img, amount=0.06),
      "speckle": add_speckle(img, var=0.03),
      "gaussian": add_gaussian(img, var=0.005)
   }
   print("Created noise versions for:", p)

median_sizes = [3,5,7,9]
alpha_sizes = [3,5]
alpha_d = 2
morph_sizes = [3,5,7]

rows = []

for p, noises in noisy_versions.items():
    orig = images[p]

    for noise_type, noisy in noises.items():
        print(f"Image: {p} | Noise: {noise_type}")

        # baseline
        base_ps = PSNR(orig, noisy, data_range=1)
        base_ss = SSIM(orig, noisy, data_range=1)
        base_eo = edge_overlap(orig, noisy)
        print("Baseline:", base_ps, base_ss, base_eo)

        for k in median_sizes:
            med = median_filter(noisy, size=k)
            ps = PSNR(orig, med, data_range=1)
            ss = SSIM(orig, med, data_range=1)
            eo = edge_overlap(orig, med)
            rows.append([p, noise_type, f"median_{k}", ps, ss, eo])
            print(f"Median {k}: PSNR={ps:.2f} SSIM={ss:.4f} Edge={eo:.3f}")

        for k in alpha_sizes:
            at = alpha_trimmed_mean((noisy*255).astype(np.uint8), size=k, d=alpha_d)/255.0
            ps = PSNR(orig, at, data_range=1)
            ss = SSIM(orig, at, data_range=1)
            eo = edge_overlap(orig, at)
            rows.append([p, noise_type, f"alpha_trim_{k}", ps, ss, eo])
            print(f"Alpha-trim {k}: PSNR={ps:.2f} SSIM={ss:.4f} Edge={eo:.3f}")

        for s in morph_sizes:
            se = morphology.square(s)
```

```
        op = morphology.opening((noisy*255).astype(np.uint8), se)
        cl = morphology.closing(op, se).astype(np.float32)/255.0

        ps = PSNR(orig, cl, data_range=1)
        ss = SSIM(orig, cl, data_range=1)
        eo = edge_overlap(orig, cl)
        rows.append([p, noise_type, f"morph_{s}", ps, ss, eo])
        print(f"Morph {s}: PSNR={ps:.2f} SSIM={ss:.4f} Edge={eo:.3f}")

    # Show best median visually
    best = None
    for k in median_sizes:
        med = median_filter(noisy, size=k)
        ss = SSIM(orig, med, data_range=1)
        if best is None or ss > best[0]:
            best = (ss, k, med)

    show(f"{p} | {noise_type}: Best Median {best[1]} (SSIM={best[0]:.4f})",
         to_uint8(orig), to_uint8(best[2]),
         "Clean", "Best Median")

df = pd.DataFrame(rows, columns=["image","noise","method","psnr","ssim","edge_overlap"])
summary = df.sort_values(["image","noise","ssim","psnr"], ascending=[True,True,False,False])
print("Top performers:")
display(summary.head(25))

print("Practical Recommendations:")
print("1) Salt-and-pepper noise → Median 3x3 or 5x5, Morphological opening+closing")
print("2) Impulse noise → Median 5x5 or Alpha-trim filter (size 5)")
print("3) Speckle noise → Alpha-trim, median 7x7; morphological less effective")
print("4) Gaussian noise → Non-linear filters less effective; small median helps but may blur edges")
print("5) Edge preservation best → Median small window (3 or 5)")
print("6) Larger window sizes remove more noise but reduce fine detail.")
```

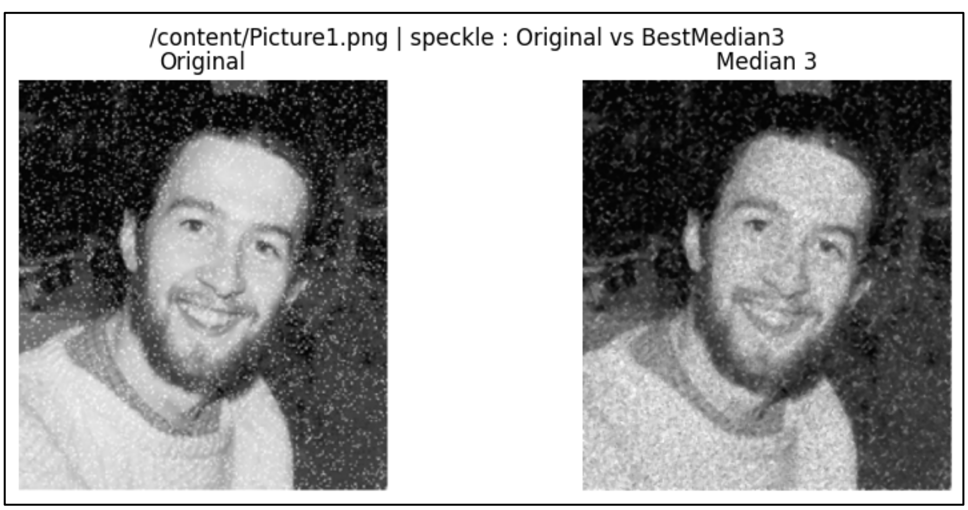
OUTPUT:

Baseline PSNR: 16.83, SSIM: 0.4157, EdgeOverlap: 0.6161
median 3: PSNR=27.32, SSIM=0.8325, EdgeOv=0.6396
median 5: PSNR=23.80, SSIM=0.6127, EdgeOv=0.2810
median 7: PSNR=22.63, SSIM=0.4818, EdgeOv=0.1347
median 9: PSNR=22.17, SSIM=0.4240, EdgeOv=0.0839
alpha-trim 3: PSNR=25.94, SSIM=0.7781, EdgeOv=0.6355
alpha-trim 5: PSNR=23.65, SSIM=0.5775, EdgeOv=0.2676
alpha-trim 7: PSNR=22.62, SSIM=0.4494, EdgeOv=0.0963
morph 3: PSNR=22.92, SSIM=0.6404, EdgeOv=0.3098
morph 5: PSNR=19.31, SSIM=0.3946, EdgeOv=0.0948
morph 7: PSNR=14.95, SSIM=0.2768, EdgeOv=0.0753

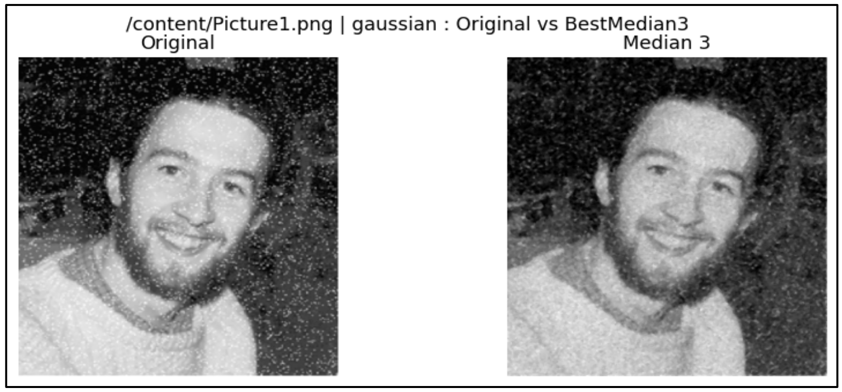
```
/tmp/ipython-input-4145038815.py:48: FutureWarning: `square` is deprecated since version 0.25 and will be removed in version 0.27. Use `skimage.morphology.footprint_rectangle` instead.
  selem = morphology.square(s)
 Best median size (by SSIM) = 3 (SSIM=0.8325)
```



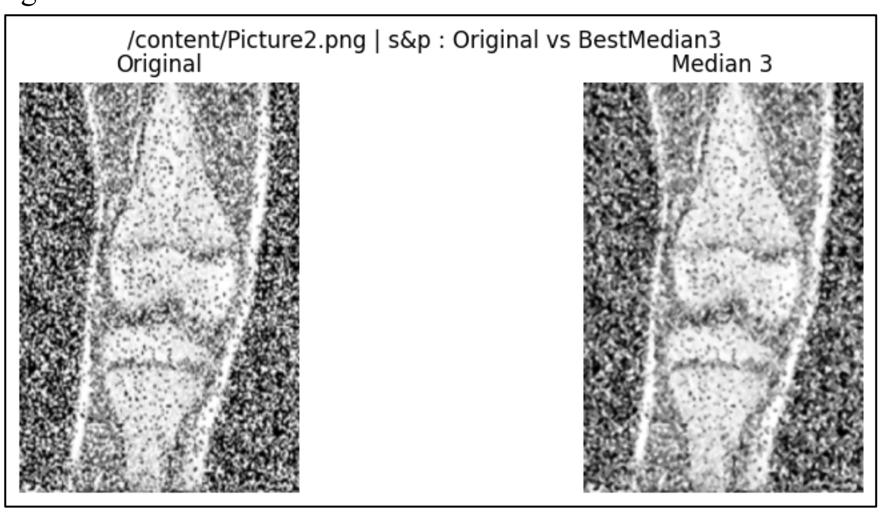

```
Baseline PSNR: 21.54, SSIM: 0.6297, EdgeOverlap: 0.7454
 median 3: PSNR=25.08, SSIM=0.6613, EdgeOv=0.5833
 median 5: PSNR=23.33, SSIM=0.5339, EdgeOv=0.2604
 median 7: PSNR=22.43, SSIM=0.4421, EdgeOv=0.1088
 median 9: PSNR=22.04, SSIM=0.3997, EdgeOv=0.0621
 alpha-trim 3: PSNR=25.50, SSIM=0.7050, EdgeOv=0.6235
 alpha-trim 5: PSNR=23.79, SSIM=0.5720, EdgeOv=0.2645
 alpha-trim 7: PSNR=22.81, SSIM=0.4604, EdgeOv=0.0787
 morph 3: PSNR=20.05, SSIM=0.5220, EdgeOv=0.3065
 morph 5: PSNR=16.78, SSIM=0.3504, EdgeOv=0.0821
 morph 7: PSNR=15.47, SSIM=0.3158, EdgeOv=0.0625
 Best median size (by SSIM) = 3 (SSIM=0.6613)
```



Baseline PSNR: 23.38, SSIM: 0.6221, EdgeOverlap: 0.7071
median 3: PSNR=25.93, SSIM=0.7217, EdgeOv=0.5836
median 5: PSNR=23.75, SSIM=0.5771, EdgeOv=0.2686
median 7: PSNR=22.78, SSIM=0.4657, EdgeOv=0.1087
median 9: PSNR=22.36, SSIM=0.4114, EdgeOv=0.0634
alpha-trim 3: PSNR=26.04, SSIM=0.7568, EdgeOv=0.6046
alpha-trim 5: PSNR=23.92, SSIM=0.5950, EdgeOv=0.2458
alpha-trim 7: PSNR=22.87, SSIM=0.4690, EdgeOv=0.0769
morph 3: PSNR=21.27, SSIM=0.5550, EdgeOv=0.3132
morph 5: PSNR=18.11, SSIM=0.3709, EdgeOv=0.0922
morph 7: PSNR=16.97, SSIM=0.3290, EdgeOv=0.0701
Best median size (by SSIM) = 3 (SSIM=0.7217)



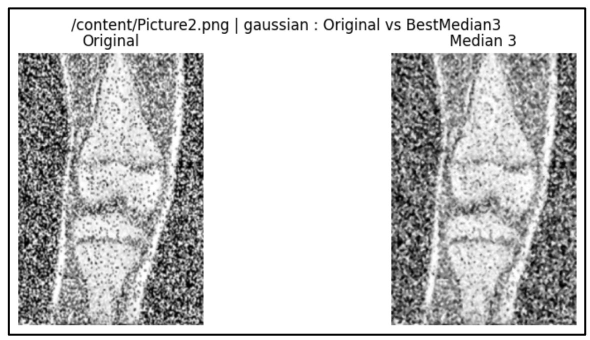
Baseline PSNR: 16.89, SSIM: 0.7391, EdgeOverlap: 0.7645
median 3: PSNR=21.69, SSIM=0.8715, EdgeOv=0.7507
median 5: PSNR=17.62, SSIM=0.6108, EdgeOv=0.4451
median 7: PSNR=15.51, SSIM=0.3374, EdgeOv=0.2340
median 9: PSNR=14.71, SSIM=0.2001, EdgeOv=0.1769
alpha-trim 3: PSNR=21.16, SSIM=0.8479, EdgeOv=0.7420
alpha-trim 5: PSNR=17.32, SSIM=0.5706, EdgeOv=0.4018
alpha-trim 7: PSNR=15.42, SSIM=0.3047, EdgeOv=0.1913
morph 3: PSNR=17.29, SSIM=0.6838, EdgeOv=0.4880
morph 5: PSNR=11.54, SSIM=0.2981, EdgeOv=0.2180
morph 7: PSNR=8.29, SSIM=0.1327, EdgeOv=0.1465
Best median size (by SSIM) = 3 (SSIM=0.8715)



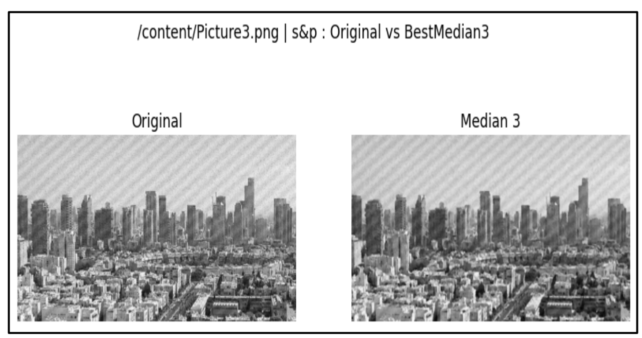
Marwadi University	**Marwari University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing(01CT0513)	**Aim: FINAL OPEN ENDED QUESTIONS**	
QUESTION 7	**Date: 28-11-2025**	**Enrollment No: 92301733054**

Baseline PSNR: 19.83, SSIM: 0.8051, EdgeOverlap: 0.7957
median 3: PSNR=20.59, SSIM=0.8125, EdgeOv=0.6877
median 5: PSNR=17.43, SSIM=0.5843, EdgeOv=0.4152
median 7: PSNR=15.48, SSIM=0.3236, EdgeOv=0.2263
median 9: PSNR=14.72, SSIM=0.1901, EdgeOv=0.1760
alpha-trim 3: PSNR=20.84, SSIM=0.8228, EdgeOv=0.7136
alpha-trim 5: PSNR=17.29, SSIM=0.5610, EdgeOv=0.4023
alpha-trim 7: PSNR=15.40, SSIM=0.3001, EdgeOv=0.1858
morph 3: PSNR=16.05, SSIM=0.5873, EdgeOv=0.4431
morph 5: PSNR=11.83, SSIM=0.2722, EdgeOv=0.2089
morph 7: PSNR=9.92, SSIM=0.1654, EdgeOv=0.1410
Best median size (by SSIM) = 3 (SSIM=0.8125)

Baseline PSNR: 23.39, SSIM: 0.9075, EdgeOverlap: 0.8494
median 3: PSNR=21.41, SSIM=0.8516, EdgeOv=0.7309
median 5: PSNR=17.63, SSIM=0.6017, EdgeOv=0.4398
median 7: PSNR=15.54, SSIM=0.3321, EdgeOv=0.2310
median 9: PSNR=14.75, SSIM=0.1963, EdgeOv=0.1732
alpha-trim 3: PSNR=21.50, SSIM=0.8557, EdgeOv=0.7523
alpha-trim 5: PSNR=17.47, SSIM=0.5815, EdgeOv=0.4133
alpha-trim 7: PSNR=15.50, SSIM=0.3118, EdgeOv=0.1847
morph 3: PSNR=17.75, SSIM=0.6707, EdgeOv=0.4846
morph 5: PSNR=12.88, SSIM=0.3299, EdgeOv=0.2274
morph 7: PSNR=10.61, SSIM=0.2091, EdgeOv=0.1553
Best median size (by SSIM) = 3 (SSIM=0.8516)

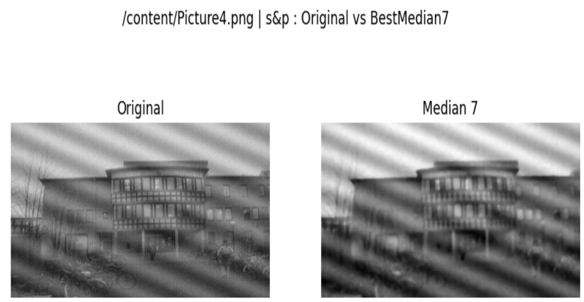


Baseline PSNR: 17.35, SSIM: 0.5159, EdgeOverlap: 0.6488
median 3: PSNR=23.96, SSIM=0.7646, EdgeOv=0.5596
median 5: PSNR=21.10, SSIM=0.5711, EdgeOv=0.2796
median 7: PSNR=19.99, SSIM=0.4703, EdgeOv=0.1732
median 9: PSNR=19.37, SSIM=0.4100, EdgeOv=0.1311
alpha-trim 3: PSNR=23.27, SSIM=0.7138, EdgeOv=0.5731
alpha-trim 5: PSNR=20.55, SSIM=0.5101, EdgeOv=0.2469
alpha-trim 7: PSNR=19.56, SSIM=0.4146, EdgeOv=0.1098
morph 3: PSNR=20.21, SSIM=0.6335, EdgeOv=0.3601
morph 5: PSNR=15.74, SSIM=0.4274, EdgeOv=0.1954
morph 7: PSNR=11.55, SSIM=0.2685, EdgeOv=0.1343
Best median size (by SSIM) = 3 (SSIM=0.7646)



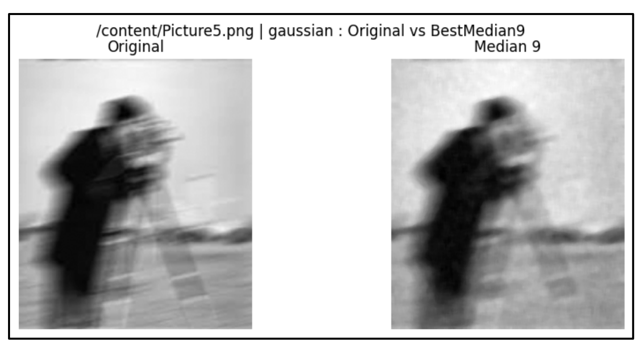
	Marwari University **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing(01CT0513)	**Aim: FINAL OPEN ENDED QUESTIONS**	
QUESTION 7	**Date: 28-11-2025**	**Enrollment No: 92301733054**

Baseline PSNR: 18.02, SSIM: 0.2821, EdgeOverlap: 0.5201
median 3: PSNR=29.31, SSIM=0.6128, EdgeOv=0.5653
median 5: PSNR=29.15, SSIM=0.5952, EdgeOv=0.2809
median 7: PSNR=29.30, SSIM=0.6267, EdgeOv=0.1355
median 9: PSNR=28.19, SSIM=0.5194, EdgeOv=0.0779
alpha-trim 3: PSNR=28.14, SSIM=0.5645, EdgeOv=0.5557
alpha-trim 5: PSNR=28.17, SSIM=0.5501, EdgeOv=0.2165
alpha-trim 7: PSNR=28.31, SSIM=0.5826, EdgeOv=0.0571
morph 3: PSNR=26.77, SSIM=0.6550, EdgeOv=0.3949
morph 5: PSNR=22.91, SSIM=0.5582, EdgeOv=0.1993
morph 7: PSNR=16.81, SSIM=0.4338, EdgeOv=0.1350
Best median size (by SSIM) = 7 (SSIM=0.6267)



Baseline PSNR: 17.50, SSIM: 0.1833, EdgeOverlap: 0.4142
median 3: PSNR=47.77, SSIM=0.9943, EdgeOv=0.7291
median 5: PSNR=45.28, SSIM=0.9883, EdgeOv=0.5753
median 7: PSNR=42.05, SSIM=0.9769, EdgeOv=0.4032
median 9: PSNR=39.20, SSIM=0.9621, EdgeOv=0.2418
alpha-trim 3: PSNR=33.22, SSIM=0.8327, EdgeOv=0.5451
alpha-trim 5: PSNR=33.91, SSIM=0.8445, EdgeOv=0.2915
alpha-trim 7: PSNR=33.68, SSIM=0.8638, EdgeOv=0.1239
morph 3: PSNR=34.86, SSIM=0.9791, EdgeOv=0.5604
morph 5: PSNR=26.37, SSIM=0.9116, EdgeOv=0.2785
morph 7: PSNR=16.88, SSIM=0.7288, EdgeOv=0.1622
Best median size (by SSIM) = 3 (SSIM=0.9943)

Baseline PSNR: 23.16, SSIM: 0.2552, EdgeOverlap: 0.4108
median 3: PSNR=30.62, SSIM=0.6464, EdgeOv=0.3674
median 5: PSNR=34.45, SSIM=0.8355, EdgeOv=0.2858
median 7: PSNR=35.88, SSIM=0.8971, EdgeOv=0.1800
median 9: PSNR=35.84, SSIM=0.9129, EdgeOv=0.1151
alpha-trim 3: PSNR=32.11, SSIM=0.7233, EdgeOv=0.3773
alpha-trim 5: PSNR=35.86, SSIM=0.8867, EdgeOv=0.2785
alpha-trim 7: PSNR=36.50, SSIM=0.9243, EdgeOv=0.1287
morph 3: PSNR=24.37, SSIM=0.6957, EdgeOv=0.3109
morph 5: PSNR=21.01, SSIM=0.7279, EdgeOv=0.1941
morph 7: PSNR=19.48, SSIM=0.7263, EdgeOv=0.1383
Best median size (by SSIM) = 9 (SSIM=0.9129)



	Marwari University **Faculty of Technology** **Department of Information and Communication Technology**
Subject: Digital Signal and Image Processing(01CT0513)	**Aim: FINAL OPEN ENDED QUESTIONS**
QUESTION 7	**Date: 28-11-2025** **Enrollment No: 92301733054**

ndex	image	noise	best_method	psnr	ssim	edge_overlap
0	/content/Picture1.png	gaussian	alpha_trim_3_d2	26.04	0.7568	0.6046
1	/content/Picture1.png	s&p	median_3	27.32	0.8325	0.6396
2	/content/Picture1.png	speckle	alpha_trim_3_d2	25.5	0.705	0.6235
3	/content/Picture2.png	gaussian	alpha_trim_3_d2	21.5	0.8557	0.7523
4	/content/Picture2.png	s&p	median_3	21.69	0.8715	0.7507
5	/content/Picture2.png	speckle	alpha_trim_3_d2	20.84	0.8228	0.7136
6	/content/Picture3.png	gaussian	alpha_trim_3_d2	23.4	0.6985	0.5784
7	/content/Picture3.png	s&p	median_3	23.96	0.7646	0.5596
8	/content/Picture3.png	speckle	alpha_trim_3_d2	22.63	0.6242	0.5731
9	/content/Picture4.png	gaussian	alpha_trim_7_d2	28.73	0.6026	0.0547
10	/content/Picture4.png	s&p	morph_openclose_3	26.77	0.655	0.3949
11	/content/Picture4.png	speckle	alpha_trim_7_d2	28.55	0.593	0.0461
12	/content/Picture5.png	gaussian	alpha_trim_7_d2	36.5	0.9243	0.1287
13	/content/Picture5.png	s&p	median_3	47.77	0.9943	0.7291
14	/content/Picture5.png	speckle	alpha_trim_7_d2	35.38	0.8991	0.1352

Obsevation:

1. Noise Identification

- Several images showed characteristics of salt-and-pepper noise, visible as random white and black pixels.
- Some images exhibited speckle noise, recognizable as granular, multiplicative noise patterns.
- A few images showed mild Gaussian-type noise, visible as smooth random variations in intensity.
- FFT and pixel-level analysis also showed impulsive peaks for salt-and-pepper cases and broader variance patterns for speckle noise.

2. Median Filtering (non-linear)

- 3×3 and 5×5 median filters performed extremely well for salt-and-pepper and impulse noise, removing isolated white/black pixels effectively.
- Smaller window sizes preserved edges better, while larger windows (7×7, 9×9) removed more noise but caused noticeable blurring and loss of fine texture.
- Median filtering consistently delivered the **best balance** between noise removal and detail retention for impulse-type noise.

3. Alpha-Trimmed Mean Filter

- Alpha-trimmed filters (size 3, 5) gave smooth and clean outputs, especially for speckle and Gaussian noise.
- They preserved more detail than large median filters because only extreme pixel values are removed.
- For salt-and-pepper noise, alpha-trim was less effective than median filtering but still reduced noise to some extent.

4. Morphological Filters (Opening + Closing)

- Morphological filtering was highly effective for salt-and-pepper noise, removing isolated bright/dark spots while maintaining shape boundaries.
- It was less effective for speckle and Gaussian noise, sometimes causing structural distortion.
- A small structuring element (3×3) preserved edges well, while larger ones (7×7) removed noise but also smoothed important details.

5. Effect of Filter Size

- **Small window sizes (3×3, 5×5)** → best edge preservation, moderate noise removal.
- **Large window sizes (7×7, 9×9)** → stronger noise removal, but blurred edges and fine details.
- The optimal size depends on noise severity:
  - Light noise → 3×3
  - Medium noise → 5×5
  - Heavy noise → 7×7

Conclusion:

From this Question, we learn that different types of noise require different non-linear filtering techniques for effective removal. Median filters work best for salt-and-pepper and impulse noise, while alpha-trimmed mean filters are more suitable for speckle and mixed noise. We also learn that filter window size plays an important role—smaller sizes preserve edges, while larger sizes remove more noise but reduce fine details. Overall, this experiment helps us understand how non-linear filters improve image quality and how choosing the right filter depends on the noise characteristics.